LLM Assignment 2 - Task 1

Optimizer Performance on Non-Convex Functions

1. Objective

The aim of this task is to implement and evaluate the five optimization algorithms on two non convex objective functions :-

1. the two-dimensional Rosenbrock function

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

where a = 1 and b = 100.

2. the simplified one-dimensional Ackley function

$$f(x) = -20exp(-0.2\sqrt{x^2}) - exp(cos(2\pi x)) + 20 + e$$

Handle x = 0 by setting f(0) = 0 or slightly exceeding zero where applicable to avoid domain errors.

The optimizers are Gradient Descent, SGD with Momentum, Adam, RMSprop, and Adagrad. Each optimizer is to be tested at learning rates 0.01, 0.05, and 0.1.

2. Functions

- **Rosenbrock Function**

  Equation: $f(x,y) = (1 - x)^2 + 100(y - x^2)^2$

  Global Minimum: (1, 1), where f = 0

  Gradient:

  - $\partial f/\partial x = 2(x - 1) - 400x(y - x^2)$
  - $\partial f/\partial y = 200(y - x^2)$

- **Simplified 1D Ackley Function**

  Equation: $f(x) = -20\exp(-0.2\sqrt{(x^2)}) - \exp(\cos(2\pi x)) + 20 + e$

  Global Optimum: x = 0 with f(0) = 0

  Implementation Note: The implementation explicitly handles x = 0.

3. Optimization Methods

Gradient Descent: $\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$.

Momentum: maintains a running velocity to smooth and accelerate parameter updates.

Adam: combines first- and second-moment estimates with bias correction and adaptive step sizes.

RMSprop: scales gradients using an exponentially weighted average of squared gradients.

Adagrad: accumulates squared gradients and progressively reduces the effective step size.

All optimizers were implemented directly using NumPy; no pre-built optimization routine was used.

4. Experimental Setup

Rosenbrock was initialized at (−1.2, 1.0), while Ackley was initialized at x = 2.0. The maximum number of iterations was 10,000. A run is marked converged when either the parameter update norm or gradient norm fell below 10^{-8}. A run is marked diverged when a non-finite parameter or objective value was produced; otherwise, reaching 10,000 iterations was reported as max_iter.

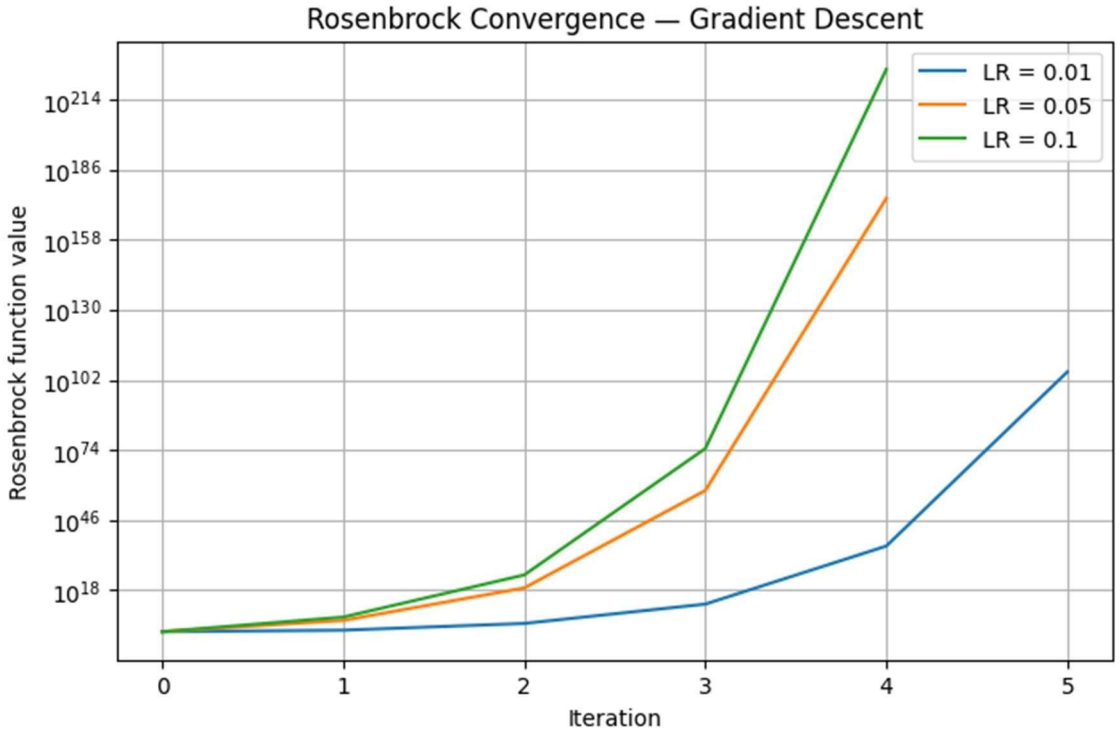
5. Rosenbrock Results

Optimizer	LR	Status	Final f	Iterations	Time (s)
Gradient Descent	0.01	Diverged	inf	6	0.000749
Gradient Descent	0.05	Diverged	inf	5	0.000413
Gradient Descent	0.10	Diverged	inf	5	0.000710
Momentum	0.01	Diverged	inf	6	0.000309
Momentum	0.05	Diverged	inf	5	0.000342
Momentum	0.10	Diverged	inf	5	0.000471
Adam	0.01	Converged	3.0965e-13	5531	0.241457
Adam	0.05	Converged	1.3855e-13	2546	0.159548
Adam	0.10	Converged	7.6961e-14	1793	0.116793
RMSprop	0.01	Max iter	2.2154e-02	10000	0.430163
RMSprop	0.05	Max iter	4.3563e-01	10000	0.517359
RMSprop	0.10	Max iter	1.0525e+00	10000	0.633585
Adagrad	0.01	Max iter	3.5736e+00	10000	0.653215
Adagrad	0.05	Max iter	5.1219e-02	10000	0.719570
Adagrad	0.10	Max iter	6.2693e-04	10000	0.761798

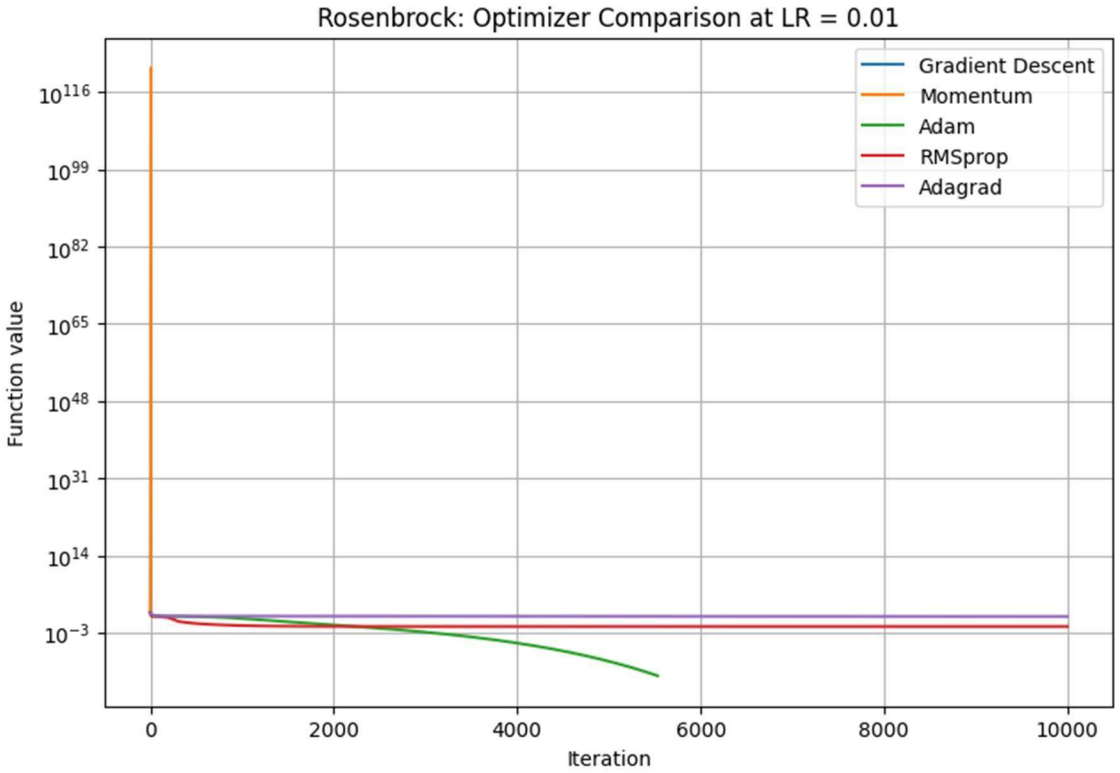
5.1 Interpretation

- We observed that Adam was the best optimizer for Rosenbrock, it converged for all three learning rates and reached values extremely close to zero. Increasing the learning rate from 0.01 to 0.1 reduced the iteration count from 5531 to 1793 and also reduced measured runtime from 0.241457 s to 0.116793 s.
- Gradient Descent and Momentum diverged for all three tested learning rates. This behaviour is consistent with the steep , narrow and curved Rosenbrock valley : The selected learning rates were too aggressive for these non-adaptive update rules from the chosen starting point.

- RMSprop remained finite but did not reach the convergence criterion within 10,000 iterations.
- Adagrad improved substantially as the learning rate increased; its best tested result was $f \approx 6.27 \times 10^{-4}$ at $\alpha = 0.1$, although it still reached the iteration limit.



Rosenbrock convergence for Gradient Descent. The large increase in objective value illustrates divergence.



Rosenbrock optimizer comparison at α = 0.01. Adam continues decreasing toward the global minimum while the other methods stagnate or diverge.

6. Ackley Results

Optimizer	LR	Status	Final f	Iterations	Time (s)
Gradient Descent	0.01	Converged	6.559645	5	0.000343
Gradient Descent	0.05	Max iter	3.271590	10000	0.759391
Gradient Descent	0.10	Max iter	3.595542	10000	0.685959
Momentum	0.01	Converged	6.559645	252	0.014630
Momentum	0.05	Max iter	20.260769	10000	0.865630
Momentum	0.10	Max iter	15.234211	10000	0.670696
Adam	0.01	Converged	6.559645	238	0.010092
Adam	0.05	Converged	6.559645	247	0.010117
Adam	0.10	Converged	6.559645	274	0.010969
RMSprop	0.01	Max iter	6.560894	10000	0.362561
RMSprop	0.05	Max iter	6.588734	10000	0.353068
RMSprop	0.10	Max iter	6.661309	10000	0.384960
Adagrad	0.01	Converged	6.559645	40	0.001437
Adagrad	0.05	Converged	6.559645	18	0.000743
Adagrad	0.10	Converged	6.559645	29	0.001039

6.1 Interpretation

The simplified Ackley function has a global minimum at $x = 0$, $f = 0$, but the experiments start from $x = 2$. Several optimizers settle near $x \approx 1.974452$ with $f \approx 6.559645$. This point is a local minimum rather than the global optimum. Therefore, a converged status only means that the selected stopping criterion was satisfied; it does not guarantee that the global minimum was found.

At $\alpha = 0.01$, Gradient Descent, Momentum, Adam, and Adagrad converge to this local minimum. Adagrad is particularly fast in iteration count, requiring 40 iterations at $\alpha = 0.01$ and only 18 iterations at $\alpha = 0.05$. Adam requires approximately 238–274 iterations across the three learning rates and remains stable.

Gradient Descent at $\alpha = 0.05$ reaches $x \approx 0.315579$ and $f \approx 3.271590$ but hits the 10,000-iteration limit. Momentum becomes less effective at the larger learning rates, reaching much higher objective values. RMSprop stays close to the initial region and reaches its iteration limit for all three learning rates.

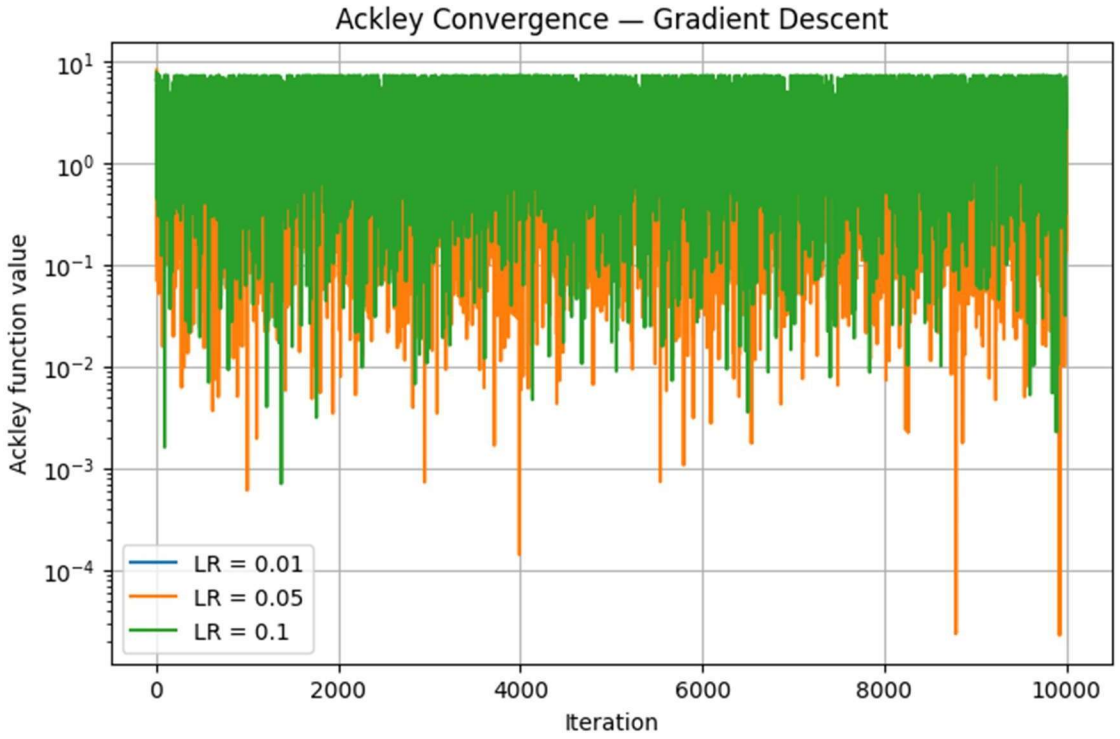


Figure 3. Ackley convergence for Gradient Descent across the three tested learning rates.

7. Fixed Learning-Rate Comparison

The optimal solution obtained by the different GD-based algorithms for a fixed learning rate. The notebook uses $\alpha = 0.01$ for the direct optimizer comparison. For Rosenbrock, Adam gives the best finite result among the five methods at this learning rate: $x \approx (0.99999944, 0.99999889)$ with $f \approx 3.10 \times 10^{-13}$.

This comparison also showed the practical effect of optimizer design : ordinary Gradient Descent and Momentum diverge while adaptive methods remain finite.

8. Effect of Learning Rate

The experiments show that the learning rate must be considered together with the optimizer. For Rosenbrock, Adam benefits strongly from increasing α: its iteration count decreases from 5531 at 0.01 to 1793 at 0.1 without sacrificing solution quality. In contrast, increasing α for RMSprop worsens the final objective value in these experiments, and Gradient Descent/Momentum remain unstable.

For Ackley, changing α does not necessarily improve the final global result because the optimizer can be attracted to a local minimum. Adam reaches essentially the same local minimum at all three learning rates. Adagrad changes the speed of convergence substantially but reaches the same local minimum.

9. Key Findings

- Adam is the most successful method for the Rosenbrock experiment among the tested configurations.
- For Rosenbrock, Adam at $\alpha = 0.1$ gives the lowest recorded objective value, approximately 7.70×10^{-14}.

- Gradient Descent and Momentum diverge on Rosenbrock at all tested learning rates.
- Adagrad at $\alpha = 0.1$ gets close to the Rosenbrock minimum but reaches the 10,000-iteration limit.
- On Ackley, several methods converge to a local minimum near $x = 1.974452$ rather than the global minimum $x = 0$.
- The results demonstrate that convergence of updates is not equivalent to finding the global optimum on a non-convex function.
- Learning rate has a strong interaction with optimizer choice; no single learning rate is universally best.

10. Conclusion

This experiment showed different numerical behaviour of first order optimizers on non-convex objectives. Rosenbrock highlighted the instability of large fixed-step updates and the advantage of adaptive optimization, with Adam reaching the global minimum to numerical precision.
Ackley experiment highlighted a different challenge : an optimizer might converge rapidly to a local minimum while remaining far from the global optimum. Overall, optimizer choice, learning rate, initialization and stopping criteria all influence the convergence speed and final solution quality.

LLM Assignment 2 - Task 2

Linear Regression Using a Multi-Layer Neural Network From Scratch

1. Objective

The aim of this task is to implement a small multi-layer neural network from scratch and use it for predicting the median house value from the California Housing dataset. The main purpose was to understand how forward propagation, backpropagation and different optimizers work without using a pre-built neural network model.

The required network uses two input features, two hidden layers and one output neuron. Gradient Descent, Momentum and Adam were tested with learning rates 0.01 and 0.001. The assignment also contains two bonus parts: adding another hidden layer and applying L2 regularization.

2. Dataset and Preprocessing

The notebook uses sklearn.datasets.fetch_california_housing only to load the specified California Housing dataset. The two input features are MedInc (Median Income) and HouseAge (House Age), and the target is MedHouseVal (Median House Value). The loaded dataset contains 20,640 samples with two input features and one target.

A manual 80/20 split is performed using a reproducible seed of 42, producing 16,512 training samples and 4,128 testing samples. Feature normalization uses only the training-set mean and standard deviation, and the same training statistics are applied to the test set. The recorded training mean is [3.8716266, 28.70724322] and the training standard deviation is [1.90161359, 12.60605401].

$$X_norm = (X - mean) / standard\ deviation$$

3. Network Architecture

The required network is:

$$\mathbf{2 \rightarrow 5 \rightarrow 3 \rightarrow 1}$$

The first hidden layer contains 5 neurons and the second hidden layer contains 3 neurons. Both hidden layers use ReLU activation. The output layer is linear because the task is regression.

Weights use He initialization, which is appropriate for ReLU hidden layers. Biases are initialized to zero.

4. Forward Propagation and Loss

Forward propagation is implemented as:

$$Z_1 = XW_1 + b_1$$

$$A_1 = \mathrm{ReLU}(Z_1)$$

$$Z_2 = A_1W_2 + b_2$$

$$A_2 = \mathrm{ReLU}(Z_2)$$

$$\hat{y} = A_2W_3 + b_3$$

The mean squared error is:

$$MSE = \text{mean}((y - \hat{y})^2)$$

5. Backpropagation

The notebook computes the MSE derivative as $2(\hat{y} - y)/m$, where m is the number of training samples. The resulting output gradient is propagated through the linear output layer and the two ReLU hidden layers. The weight gradients are computed with matrix multiplications and the bias gradients are obtained by summing over the training examples. The implementation therefore performs the complete backward pass manually.

6. Optimizers

Three required optimizers are implemented:

- **Gradient Descent:** $\theta_{t+1} = \theta_t - \alpha g_t$.
- **Momentum:** $v_t = \beta v_{t-1} + g_t$, followed by $\theta_{t+1} = \theta_t - \alpha v_t$.
- **Adam:** Maintains first- and second-moment estimates, applies bias correction, and uses adaptive parameter updates.

Each experiment is trained for **1,000 epochs**. For a fair comparison, every optimizer–learning-rate run starts from the same initialization using **random seed 42**.

7. Required Experiments and Results

Optimizer	Learning Rate	Train MSE	Test MSE	Time (s)
Gradient Descent	0.01	0.677049	0.690856	5.6372
Gradient Descent	0.001	1.391723	1.403645	5.8395
Momentum	0.01	0.616038	0.626382	3.0442
Momentum	0.001	0.681942	0.695260	2.9460
Adam	0.01	0.629863	0.640979	2.9620
Adam	0.001	0.672282	0.674156	4.4078

7.1 Required Results — Discussion

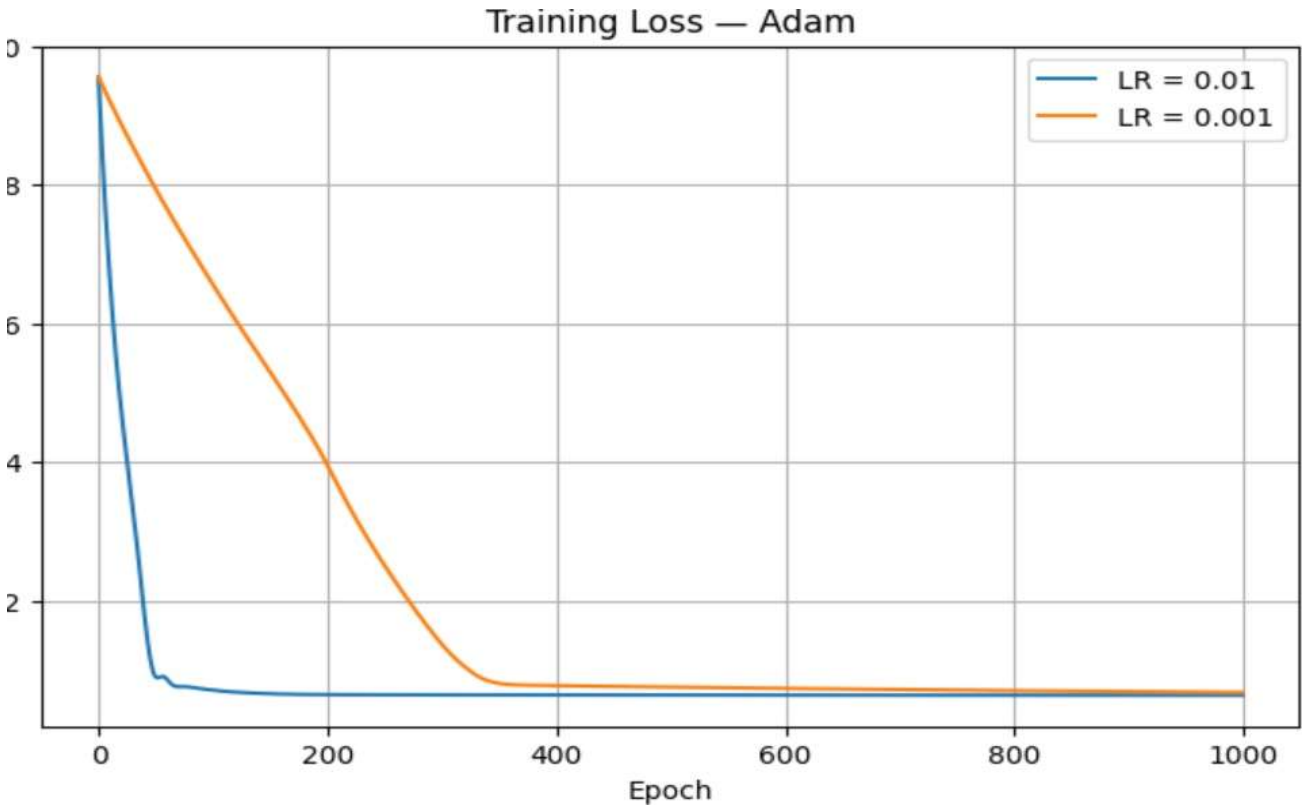
Among the required experiments, Momentum with learning rate 0.01 gives the lowest test MSE, 0.626382. Adam with learning rate 0.01 is the second-best required configuration with test MSE 0.640979. Gradient Descent at learning rate 0.001 gives the highest test MSE, 1.403645.

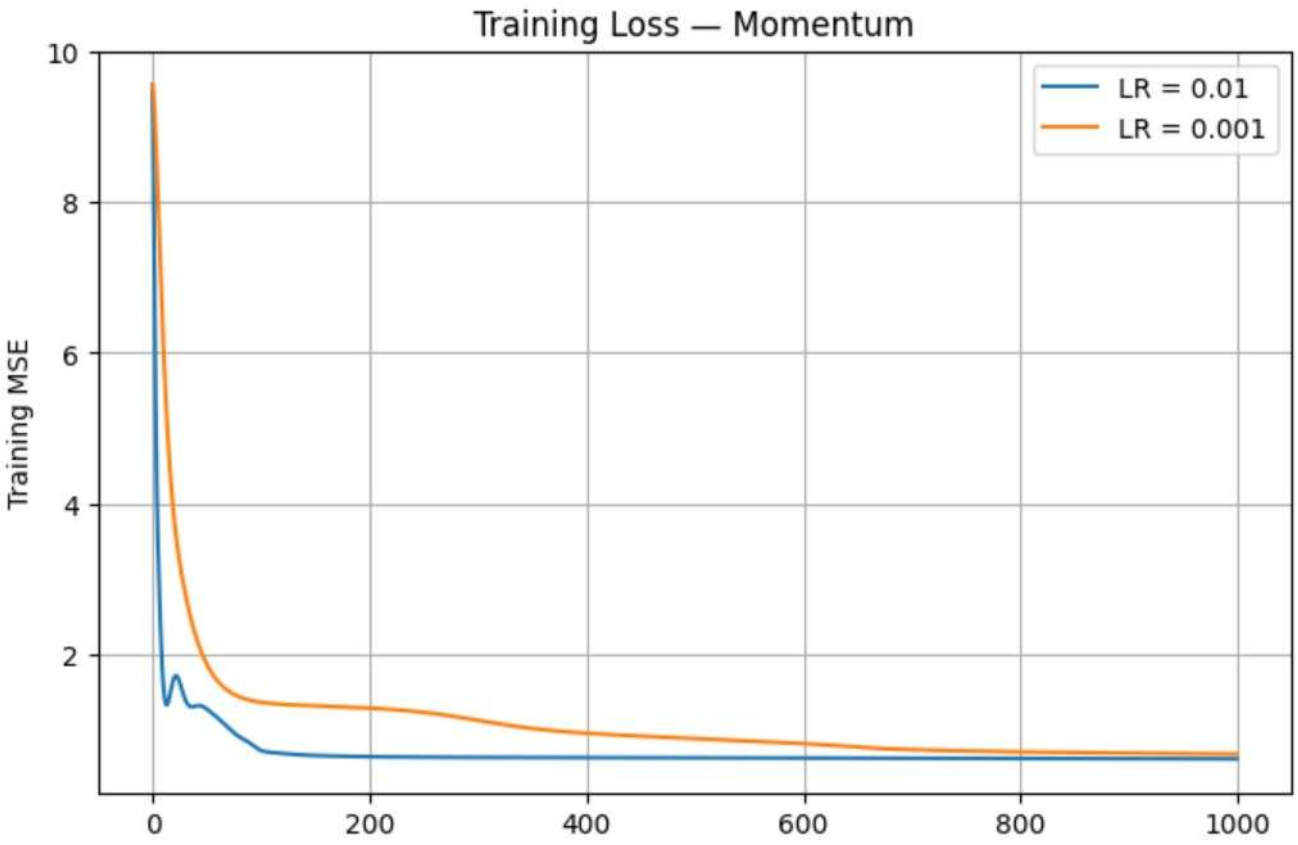
For all three optimizers in this experiment, learning rate 0.01 produces a lower test MSE than 0.001. The results therefore show that the larger tested learning rate is more effective for this particular network, dataset split, initialization, and 1,000-epoch training setup.

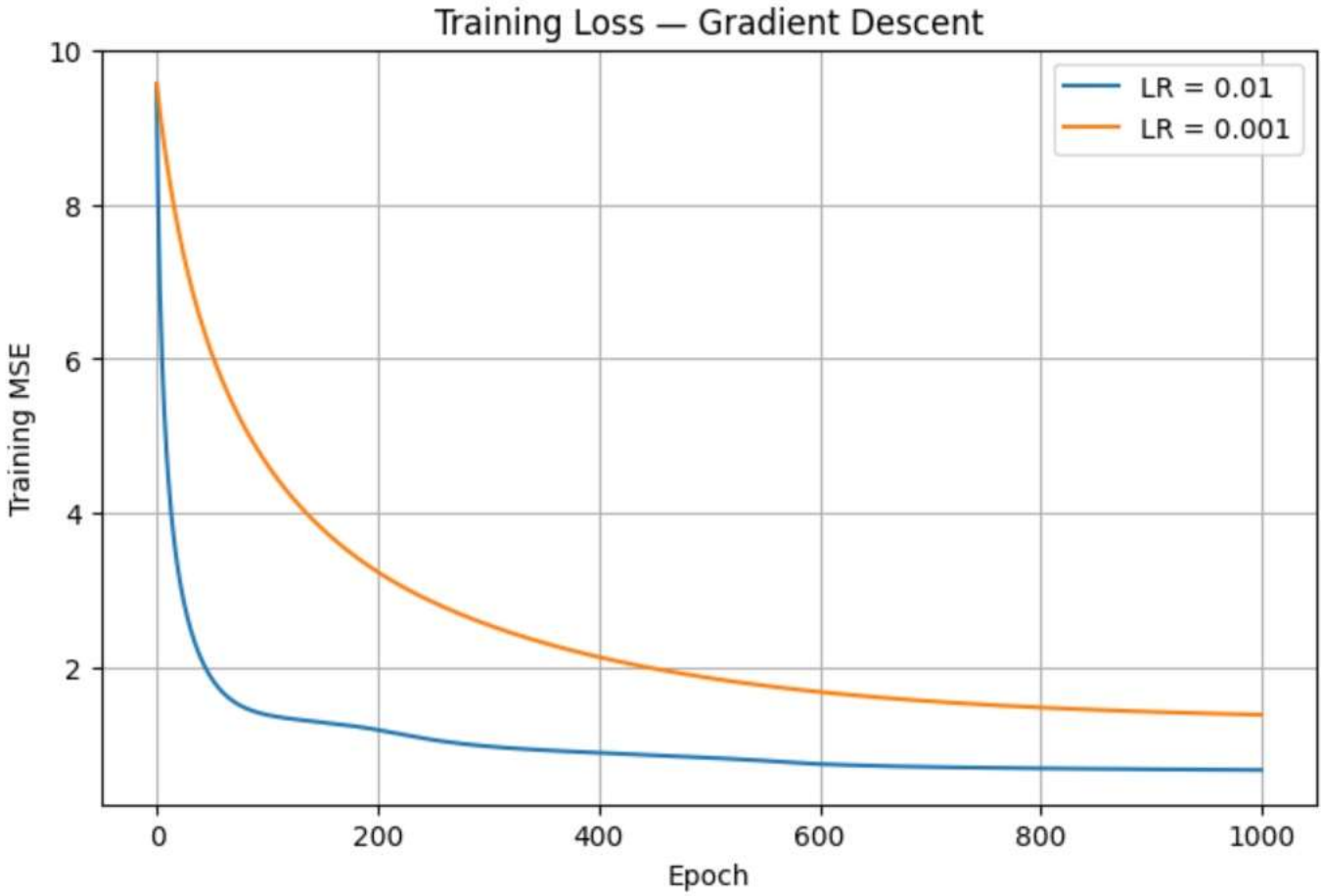
Momentum at 0.01 also has the lowest measured training time among the required runs except for the very similar Adam 0.01 run. The timing values are environment-dependent and should primarily be interpreted as measurements for this execution rather than universal performance benchmarks.

8. Training-Loss Curves and Prediction Analysis

The notebook records the training MSE at every epoch and produces training-loss curves for the required experiments. It also evaluates the trained models on the held-out test set and generates predicted-versus-actual plots. These plots provide visual evidence of how the loss changes during training and how closely predictions follow the observed target values.


Training Loss — Adam
LR = 0.01
LR = 0.001
0
200
400
600
800
1000
Epoch


Training Loss — Momentum
LR = 0.01
LR = 0.001
Training MSE
10
8
6
4
2
0
200
400
600
800
1000


Training Loss — Gradient Descent
LR = 0.01
LR = 0.001
Training MSE
Epoch
10
8
6
4
2
0
200
400
600
800
1000

9. Bonus 1 — Additional Hidden Layer

Bonus 1 adds a third hidden layer containing 2 neurons, changing the architecture from 2 → 5 → 3 → 1 to 2 → 5 → 3 → 2 → 1. The additional hidden layer also uses ReLU. The notebook compares this deeper architecture using Adam at both required learning rates.

Architecture	LR	Train MSE	Test MSE
Original 2→5→3→1	0.01	0.629863	0.640979
Bonus 2→5→3→2→1	0.01	0.619861	0.633882
Original 2→5→3→1	0.001	0.672282	0.674156
Bonus 2→5→3→2→1	0.001	0.667749	0.681811

At learning rate 0.01, the additional hidden layer improves test MSE from 0.640979 to 0.633882. At learning rate 0.001, however, the deeper architecture gives a higher test MSE, 0.681811, compared with 0.674156 for the original architecture. Thus, the additional layer helps at 0.01 in this experiment but does not universally improve generalization.

10. Bonus 2 — L2 Regularization

Bonus 2 adds L2 regularization to the original network. The notebook compares λ = 0 (no regularization) with λ = 0.001. The regularization penalty is applied to the network weights; biases are not regularized.

Lambda	Train MSE	Test MSE	Time (s)
λ = 0.0	0.672282	0.674156	3.8427
λ = 0.001	0.665852	0.668685	2.9786

With λ = 0.001, the training MSE decreases from 0.672282 to 0.665852 and the test MSE decreases from 0.674156 to 0.668685. Within this experiment, the tested L2 regularization therefore produces a modest improvement in both training and test MSE.

11. Overall Comparison

Experiment	Best Test MSE	Configuration
Required experiments	0.626382	Momentum, LR = 0.01
Bonus 1	0.633882	2→5→3→2→1, Adam, LR = 0.01
Bonus 2	0.668685	L2 λ = 0.001, Adam, LR = 0.001

The lowest test MSE recorded in the notebook is 0.626382 from the required Momentum experiment at learning rate 0.01. The deeper Bonus 1 architecture achieves 0.633882 at 0.01, improving on the corresponding original Adam result of 0.640979. The L2 experiment reduces the Adam 0.001 baseline test MSE from 0.674156 to 0.668685.

12. Conclusion

The assignment demonstrates that a neural network for regression can be constructed entirely from basic numerical operations. The required 2 → 5 → 3 → 1 network successfully performs forward propagation, backpropagation, and optimization without using a pre-built neural-network or optimizer library. Among the required configurations, Momentum with learning rate 0.01 achieves the best test MSE of 0.626382. The bonus experiments show that architecture depth and regularization can affect generalization differently depending on the learning rate and training configuration. In particular, the additional hidden layer improves the 0.01 Adam result, while L2 regularization with λ = 0.001 improves the tested Adam 0.001 result.

13. Implementation Verification

The notebook records the dataset loading, manual split, training-only normalization, required network architecture, manual backpropagation, three required optimizers, both learning rates, 1,000 epochs, training loss tracking, execution time, test MSE, training-loss plots, predicted-versus-actual plots, and both requested bonus experiments. The executed notebook also reports the corresponding numerical results used in this report.